

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA1402

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks:50

Annexure A

DATA INTERPRETATION

Code of Conduct:

I AL NAJEEB 192424325(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

DATA INTERPRETATION

Q1. Intermediate Code Analysis and Optimization

Given the three-address code:

$t1 = x + y$

$t2 = t1 * z$

$t3 = t2 / w$

$t4 = t3 - p$

Analyze the sequence of operations and construct the equivalent expression tree. Evaluate how temporary variables improve intermediate code generation and reduce repeated computations.

Q2. Symbol Table and Memory Allocation

Given the following symbol table entries:

Variable Data Type Size (Bytes)

p char 1

q double 8

r int 4

s float 4

Interpret how the compiler allocates memory for these variables and calculate the total memory required. Analyze how alignment and data types affect memory organization.

Q3. Optimization in Intermediate Code

Given the intermediate code:

$t1 = m * n$

$t2 = p + q$

$t3 = m * n$

$t4 = t2 - t3$

Trace the execution step-by-step and identify optimization opportunities such as common subexpression elimination and redundant computation removal. Explain how optimization improves execution efficiency.

Q4. Control Flow and Boolean Expression Handling

Given the Boolean expression:

$\text{if } (x > y \parallel p < q)$

Intermediate code:

if $x > y$ goto L1

if $p < q$ goto L1

goto L2

L1: ...

Interpret the control flow of the given intermediate representation and explain how short-circuit evaluation minimizes unnecessary condition checking during execution.

Q5. Backpatching and Flow Control

Given:

Instruction Target

200: if x==y goto _

201: goto _

Backpatch lists:

- truelist = {200}
- falselist = {201}

Analyze the role of backpatching in intermediate code generation. Demonstrate how target addresses are updated during control flow handling and explain its importance in Boolean expression translation.

Q1. Intermediate Code Analysis and Optimization

QUESTION

Given the three-address code:

$t1 = x + y$

$t2 = t1 * z$

$t3 = t2 / w$

$t4 = t3 - p$

Analyze the sequence of operations and construct the equivalent expression tree. Evaluate how temporary variables improve intermediate code generation and reduce repeated computations.

ANSWER

Step 1 — Given Three-Address Code

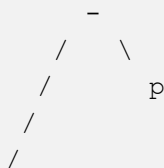
```
t1 = x + y      ; instruction 1
t2 = t1 * z     ; instruction 2
t3 = t2 / w     ; instruction 3
t4 = t3 - p     ; instruction 4
```

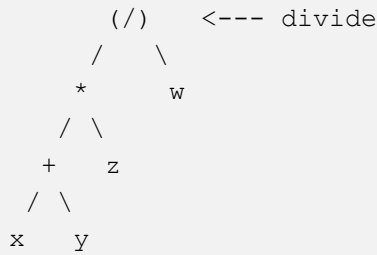
Step 2 — Expression Tree Construction

An expression tree (also called a syntax tree for expressions) represents the hierarchical structure of the computation. Each internal node is an operator and each leaf is an operand (variable or constant). The tree is built by tracing the dependency chain of the temporary variables from the bottom up.

The expression represented by the four TAC instructions is: $t4 = ((x + y) * z) / w - p$

Expression Tree (read bottom-to-top):





Level-by-level (bottom to top):

Leaf nodes : x, y, z, w, p

Level 1 (t1) : + (x, y)

Level 2 (t2) : * (t1, z) = * ((x+y), z)

Level 3 (t3) : / (t2, w) = / (((x+y)*z), w)

Level 4 (t4) : - (t3, p) = (((x+y)*z)/w) - p

Step 3 — Evaluation of Temporary Variables

Temporary variables (t1, t2, t3, t4) serve as named storage locations for intermediate computed results within a single expression. Their role in intermediate code generation is critical for several reasons.

First, they decompose every complex expression into a sequence of simple binary operations, each involving exactly two operands and one operator. This three-address form is the universal input format expected by back-end optimisers and code generators. Every machine architecture, whether RISC or CISC, can directly map these binary operations onto its native instruction set.

Second, temporary variables allow the compiler to detect and eliminate repeated computations through common subexpression elimination. If the same sub-expression such as (x + y) appears in multiple places in a larger program, the compiler needs to compute it only once, store the result in a temporary, and reuse the temporary wherever the sub-expression occurs. This eliminates redundant arithmetic instructions at runtime.

Third, temporaries enable register allocation algorithms to determine exactly which values need to be in registers at any given point in the program. The live range of a temporary variable spans from its definition to its last use, and register allocators assign registers to temporaries based on these live ranges to minimise memory traffic.

Step 4 — Step-by-Step Execution Trace

Step	Instruction	Operation	Result Stored In
1	t1 = x + y	Add x and y	t1
2	t2 = t1 * z	Multiply t1 (= x+y) with z	t2

3	$t3 = t2 / w$	Divide $t2 (= (x+y)*z)$ by w	$t3$
4	$t4 = t3 - p$	Subtract p from $t3$	$t4$ (final result)

Step 5 — How Optimization Reduces Repeated Computations

If this expression appears more than once in the source program, say the same formula is computed again later, the compiler's common subexpression elimination pass recognises that $t1 = x + y$ has already been computed and x, y have not been modified since. It then replaces the second computation of $x + y$ with a direct reference to $t1$, eliminating a redundant addition. This analysis is made possible by the temporary variable naming convention which gives each computed value a unique, traceable name in the intermediate representation.

The expression tree also reveals that the four operations must be performed in strict dependency order: $t1$ must be computed before $t2$, $t2$ before $t3$, and $t3$ before $t4$. This ordering is the data-flow dependency chain of the expression, and the compiler's scheduler respects it while also looking for opportunities to execute independent sub-expressions in parallel on processors with multiple execution units.

Q2. Symbol Table and Memory Allocation

QUESTION

Given the following symbol table entries:

Variable Data Type Size (Bytes)

p char 1
q double 8
r int 4
s float 4

Interpret how the compiler allocates memory for these variables and calculate the total memory required. Analyze how alignment and data types affect memory organization.

ANSWER

Step 1 — Symbol Table Data

Variable	Data Type	Size (Bytes)	Alignment Requirement
----------	-----------	--------------	-----------------------

p	char	1	1-byte boundary
q	double	8	8-byte boundary
r	int	4	4-byte boundary
s	float	4	4-byte boundary

Step 2 — Memory Alignment Rules

Modern processors access memory most efficiently when values are stored at addresses that are multiples of their size. This is the principle of natural alignment. A char (1 byte) can be stored at any address. An int (4 bytes) must be stored at an address divisible by 4. A double (8 bytes) must be stored at an address divisible by 8. A float (4 bytes) must be stored at an address divisible by 4. When the compiler lays variables out in memory sequentially, it inserts padding bytes between variables to satisfy alignment requirements.

Step 3 — Memory Layout Without Alignment Padding

If variables are packed with no padding (as in a C struct with `__attribute__((packed))`):

```
Address 0 : p (char, 1 byte) -> occupies byte 0
Address 1 : q (double, 8 bytes) -> occupies bytes 1-8 [MISALIGNED]
Address 9 : r (int, 4 bytes) -> occupies bytes 9-12 [MISALIGNED]
Address 13 : s (float, 4 bytes) -> occupies bytes 13-16 [MISALIGNED]

Total (no padding) = 1 + 8 + 4 + 4 = 17 bytes
```

Misaligned accesses cause the processor to issue multiple memory bus transactions to assemble a single value, slowing execution. Some architectures (e.g., ARM strict mode) raise a hardware fault on misaligned access.

Step 4 — Memory Layout With Alignment Padding (Standard Compiler Behaviour)

The compiler reorders or pads variables to satisfy alignment. Assuming layout in declaration order p, q, r, s with padding:

```
Address 0 : p (char, 1 byte )
Address 1-7: [7 bytes padding] -> align q to 8-byte boundary
Address 8 : q (double, 8 bytes ) -> bytes 8-15
Address 16 : r (int, 4 bytes ) -> bytes 16-19 (already 4-byte aligned)
Address 20 : s (float, 4 bytes ) -> bytes 20-23 (already 4-byte aligned)

Total memory with padding = 1 + 7 (pad) + 8 + 4 + 4 = 24 bytes
```

Step 5 — Memory Calculation Summary

Variable	Offset (bytes)	Size (bytes)	Padding After (bytes)	End Address
p	0	1	7 (to align q)	0
q	8	8	0	15
r	16	4	0	19
s	20	4	0	23

Total memory allocated by compiler = 24 bytes

Actual data bytes = 1 + 8 + 4 + 4 = 17 bytes

Padding bytes inserted = 7 bytes (between p and q)

Step 6 — How Alignment and Data Types Affect Memory Organisation

The data type determines both the size and the required alignment boundary for each variable. Larger types such as double require stricter alignment (8-byte boundaries), which forces the compiler to insert significant padding after smaller types that precede them in the layout. Programmers can minimise wasted padding by ordering struct members from largest to smallest: declaring q (double), r (int), s (float), p (char) in that order would produce a layout with zero padding, using only 17 bytes. The compiler's symbol table records the offset of each variable from the start of the data region, which is used during code generation to emit the correct load and store instructions with the appropriate base address and offset.

Q3. Optimization in Intermediate Code

QUESTION

Given the intermediate code:

$$t1 = m * n$$

$$t2 = p + q$$

$$t3 = m * n$$

$$t4 = t2 - t3$$

Trace the execution step-by-step and identify optimization opportunities such as common subexpression elimination and redundant computation removal. Explain how optimization improves execution efficiency.

ANSWER

Step 1 — Original Intermediate Code

```

t1 = m * n      ; instruction 1 – compute m*n, store in t1
t2 = p + q      ; instruction 2 – compute p+q, store in t2
t3 = m * n      ; instruction 3 – compute m*n AGAIN, store in t3
t4 = t2 - t3     ; instruction 4 – compute t2 minus t3

```

Step 2 — Step-by-Step Execution Trace

Step	Instruction	Computed Value	Observation
1	t1 = m * n	$m \times n$	First computation of $m*n$
2	t2 = p + q	$p + q$	Independent computation
3	t3 = m * n	$m \times n$ (again)	REDUNDANT — same as t1
4	t4 = t2 - t3	$(p+q) - (m*n)$	Final result

Step 3 — Identifying Optimization Opportunities

Opportunity 1: Common Subexpression Elimination (CSE). The expression $m * n$ is computed identically at instruction 1 (producing t1) and again at instruction 3 (producing t3). Since neither m nor n is modified between instructions 1 and 3, the value of $m * n$ is the same at both points. The second computation is therefore a common subexpression that can be eliminated.

Opportunity 2: Dead Code Elimination. After CSE, t1 holds the value of $m * n$ and t3 is replaced by t1. If t1 is used only in this chain (i.e., only in the expression that replaces t3), and t1 is not used anywhere else in the program, the assignment to t1 itself becomes the only reference. After replacing t3 with t1 at instruction 4, t3 is no longer referenced anywhere, making instruction 3 dead code that can be removed entirely.

Step 4 — Applying Common Subexpression Elimination

The compiler recognises that $t3 = m * n$ is the same as $t1 = m * n$ computed two instructions earlier, with no intervening assignments to m or n. It replaces t3 with t1 throughout the remaining code:

BEFORE CSE:	AFTER CSE:
t1 = m * n	-> t1 = m * n
t2 = p + q	-> t2 = p + q
t3 = m * n	-> [ELIMINATED – t3 replaced by t1]
t4 = t2 - t3	-> t4 = t2 - t1

Step 5 — Optimised Intermediate Code

```

t1 = m * n      ; compute m*n once
t2 = p + q      ; compute p+q

```

```
t4 = t2 - t1 ; subtract - uses t1 instead of redundant t3
```

The optimised code has 3 instructions instead of 4. The multiplication $m * n$ is now performed only once. The temporary variable $t3$ is eliminated entirely, reducing the number of temporaries from 4 to 3. This frees up one register in the register allocation phase, potentially reducing memory spill operations.

Step 6 — How Optimization Improves Execution Efficiency

Metric	Before Optimization	After Optimization	Improvement
Number of instructions	4	3	25% fewer instructions
Number of multiplications	2 ($m*n$ twice)	1 ($m*n$ once)	50% fewer multiplications
Temporary variables	4 ($t1-t4$)	3 ($t1,t2,t4$)	1 fewer register needed
Execution cycles (approx)	~4 cycles	~3 cycles	Faster execution

Multiplication is one of the more expensive arithmetic operations on most processor architectures. Eliminating one multiplication directly reduces the CPU cycle count for this code segment. In inner loops that execute this code millions of times, the savings compound significantly, producing measurable improvements in overall program runtime. CSE is therefore one of the most impactful local optimisations performed by production compilers.

Q4. Control Flow and Boolean Expression Handling

QUESTION

Given the Boolean expression:

$if (x > y \parallel p < q)$

Intermediate code:

$if x > y goto L1$

$if p < q goto L1$

$goto L2$

$L1: \dots$

Interpret the control flow of the given intermediate representation and explain how short-circuit evaluation minimizes unnecessary condition checking during execution.

ANSWER

Step 1 — Source Expression and Generated Intermediate Code

Source: `if (x > y || p < q)`

Generated TAC:

```
100: if x > y goto L1    ; evaluate left operand of OR
101: if p < q goto L1    ; evaluate right operand of OR
102: goto L2             ; both conditions false – go to else
103: L1: ...             ; true branch
104: L2: ...             ; false branch
```

Step 2 — Control Flow Graph

```
[Start]
|
v
[100: if x > y goto L1]
|  TRUE -----> [L1: true branch]
|  FALSE
v
[101: if p < q goto L1]
|  TRUE -----> [L1: true branch]
|  FALSE
v
[102: goto L2]
|
v
[L2: false branch]
```

Step 3 — Interpreting the Control Flow

Instruction 100 evaluates the first condition $x > y$. If this condition is true, control jumps immediately to L1, the true branch, without evaluating the second condition $p < q$ at all. This is the short-circuit behaviour of the OR operator. Since OR is true as soon as any one operand is true, there is no need to evaluate remaining operands once a true operand is found.

Instruction 101 is reached only if $x > y$ was false. It then evaluates $p < q$. If this condition is true, control jumps to L1. If this condition is also false, the overall OR expression is false and control falls through to instruction 102.

Instruction 102 is reached only when both conditions are false, meaning the entire OR expression evaluates to false. The unconditional goto L2 directs execution to the false branch of the if statement.

Step 4 — Short-Circuit Evaluation Analysis

Case	$x > y$	$p < q$	Instructions Executed	Result
------	---------	---------	-----------------------	--------

Case 1	TRUE	ANY	100 only (1 comparison)	Jump to L1 (true)
Case 2	FALSE	TRUE	100, 101 (2 comparisons)	Jump to L1 (true)
Case 3	FALSE	FALSE	100, 101, 102 (2 comparisons + goto)	Jump to L2 (false)

Step 5 — How Short-Circuit Evaluation Minimises Condition Checking

In Case 1, where $x > y$ is true, only one comparison instruction executes. The second comparison $p < q$ is completely skipped. This saves one comparison operation per execution of this code path. In programs where the first condition is frequently true (for example in a search loop where a match is found early), the short-circuit skips the second comparison for every successful match, potentially saving thousands of comparisons per program run.

Short-circuit evaluation is also semantically important when the second condition involves a side effect or a potentially invalid operation. For example, if $p < q$ were replaced by $(q \neq 0 \ \&\& \ p/q < 1)$, short-circuiting would prevent the division by zero when q is zero on the path where $x > y$ is already true. The compiler's intermediate code faithfully implements this semantic by ordering the conditional jumps to match the left-to-right evaluation of the OR operator, emitting the left operand's check first and short-circuiting to the true label on success.

For AND expressions ($\&\&$), the short-circuit logic is symmetric: the compiler emits a jump to the false label on the first false operand, skipping evaluation of remaining operands. This produces efficient code for compound conditions while preserving the precise evaluation semantics specified by the language standard.

Q5. Backpatching and Flow Control

QUESTION

Given:

Instruction Target

200: *if* $x == y$ *goto* ____

201: *goto* ____

Backpatch lists:

truelist = {200}

```
falselist = {201}
```

Analyze the role of backpatching in intermediate code generation. Demonstrate how target addresses are updated during control flow handling and explain its importance in Boolean expression translation.

ANSWER

Step 1 — Understanding the Given State

During single-pass intermediate code generation, the compiler emits jump instructions before it knows the target address. This happens because the target label (such as the start of the then-branch or the else-branch) has not yet been emitted when the conditional jump is generated. Backpatching is the technique of maintaining lists of incomplete jump instructions and filling in their target addresses when the target becomes known.

```
Instruction 200: if x == y goto ____    <- target unknown at emission time
Instruction 201: goto ____              <- target unknown at emission time

truelist  = {200}    <- instruction 200 should jump to the TRUE target
falselist = {201}    <- instruction 201 should jump to the FALSE target
```

Step 2 — Role of Backpatching in Intermediate Code Generation

Backpatching enables single-pass code generation for Boolean expressions and control flow statements. Without backpatching, the compiler would need a two-pass approach: the first pass generates code with placeholder targets, and the second pass fills in the actual addresses. Backpatching eliminates the need for a second pass by deferring the target address fill-in to the precise moment when the target label is created, doing it in the same forward pass.

The compiler maintains two lists for each Boolean expression sub-tree being processed: the `truelist` contains the addresses of all jump instructions that should jump to the expression's true exit, and the `falselist` contains the addresses of all jump instructions that should jump to the expression's false exit. These lists are propagated and merged as the parser combines sub-expressions using AND and OR operators, following the SDD rules for Boolean expression code generation.

Step 3 — Backpatching Demonstration

Suppose the if-statement is: `if (x == y) then S1 else S2`. The compiler generates the following sequence:

```
--- Code generated for condition (x == y) ---
```

```

200: if x == y goto ____    <- truelist  = {200}
201: goto ____              <- falselist = {201}

--- Parser now sees the "then" keyword ---
--- newlabel() generates label L1 for S1 (address 202) ---
--- backpatch(truelist, 202) fills address 202 into instruction 200 ---

Result after backpatch:
200: if x == y goto 202     <- filled in
201: goto ____             <- still pending (falselist)

--- Compiler generates code for S1 ---
202: ... (S1 code) ...
203: goto ____             <- new jump to skip else-branch; add to
S1.nextlist

--- Parser now sees "else" keyword ---
--- newlabel() generates label L2 for S2 (address 204) ---
--- backpatch(falselist, 204) fills address 204 into instruction 201 ---

Result after backpatch:
200: if x == y goto 202     <- filled (true branch)
201: goto 204               <- filled (false branch)

--- Compiler generates code for S2 ---
204: ... (S2 code) ...
205: ... (code after if-else) ...

--- backpatch(S1.nextlist, 205) fills 205 into instruction 203 ---
203: goto 205               <- filled (skip else, go to end)

```

Step 4 — Final Fully-Patched Code

Address	Instruction	Target Status
200	if x == y goto 202	Backpatched — truelist filled with 202
201	goto 204	Backpatched — falselist filled with 204
202	S1 code starts	Label L1 — true branch entry
203	goto 205	Backpatched — skip else, jump to end
204	S2 code starts	Label L2 — false branch entry
205	Code after if-else	End label

Step 5 — Importance of Backpatching in Boolean Expression Translation

Backpatching is particularly important for compound Boolean expressions built using AND and OR operators. When two sub-expressions E1 and E2 are combined as E1 OR E2, the truelist of the combined expression is the union of the truelists of E1 and E2, and the falselist of the combined expression is the falselist of E2 alone. This list-merging operation is done symbolically during the bottom-up parse without knowing the actual addresses, and the entire set of pending jumps is resolved in one backpatch call when the final target label is created. Without backpatching, each Boolean operator would require a separate forward pass to resolve targets, making the compiler significantly more complex and slower.

Backpatching also ensures that the generated intermediate code for Boolean expressions is correct by construction: every conditional jump is eventually patched to a valid target address before code generation completes. The compiler never leaves a placeholder unresolved in the final output. This guarantee is maintained by the SDD rules for Boolean expressions, which ensure that every truelist and falselist is consumed by exactly one backpatch call before the corresponding code region is considered complete.

Step 6 — Summary of Backpatching Mechanism

Concept	Description
truelist	List of instruction addresses whose jump target is the TRUE exit of the Boolean expression
falselist	List of instruction addresses whose jump target is the FALSE exit of the Boolean expression
backpatch(list, addr)	Fills the target field of every instruction in list with the address addr
newlabel()	Generates the next available label address (the address of the next instruction to be emitted)
Merge	Union of two lists, used when combining sub-expressions under OR (truelist) or AND (falselist)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand